Universidad Nacional del Este
Facultad Politécnica

Análisis de Sistemas

Matemática IV — Cálculo Numérico

INFORME TÉCNICO DEL TRABAJO PRÁCTICO GRUPAL

Sistema de Cálculo Numérico: Método de la Secante

Grupo 1 — Secante

INTEGRANTES

COMPLETAR: Nombre Apellido
COMPLETAR: Nombre Apellido
COMPLETAR: Nombre Apellido
COMPLETAR: Nombre Apellido

DOCENTE

Prof. Lic. Alicia Castillo de Dechia

4.º semestre — 2026
Ciudad del Este, Paraguay

Informe técnico — Sistema de Cálculo Numérico: Método de la Secante

> Sigue el orden que pide la guía del trabajo práctico (puntos 7, 8, 9, 11 y 16).

Índice

1. Tecnologías utilizadas
2. Fundamento matemático
3. Diseño del sistema
4. Implementación
5. Validación de datos
6. Casos de prueba y resultados
7. Funcionalidades adicionales
8. Manual de uso

1. Tecnologías utilizadas

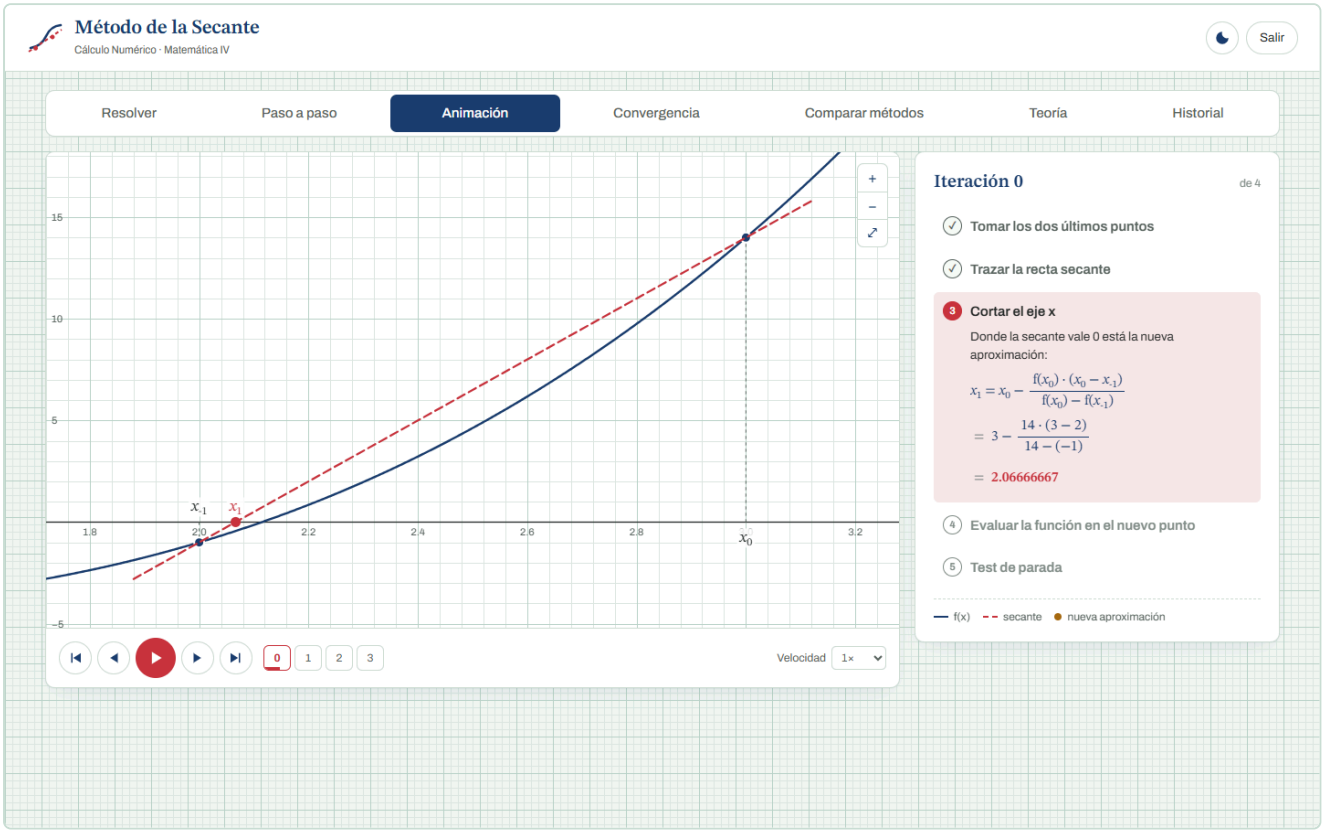
Aspecto	Elección
Lenguaje del método	**Python 3.12**: dos archivos, `inicio.py` y `metodos.py`
Interfaz	**HTML + CSS + JavaScript**, sin frameworks
Servidor	`http.server` de la biblioteca estándar de Python
Ejecutable	**PyInstaller**: genera `MetodoSecante.exe`
Pruebas	`unittest` (biblioteca estándar)
IDE	Visual Studio Code
Control de versiones	Git + GitHub

Bibliotecas de Python usadas: `math` (seno, coseno, logaritmo... para calcular f(x)), `json` y `http.server` (comunicación con la página), `webbrowser` (abrir el navegador). **El método de la secante no usa ninguna biblioteca:** la fórmula, las iteraciones, el cambio de signo y el test de parada están programados en `metodos.py`. Python solo se usa como "calculadora" para obtener f(x), como permite la consigna ("como apoyo").

2. Fundamento matemático

2.1 Concepto

El método de la secante busca una raíz de f(x) = 0. Parte de dos valores, **a** y **b**, traza la recta (secante) que pasa por (a, f(a)) y (b, f(b)) y toma como nuevo valor el punto donde esa recta corta al eje x. Después repite el proceso con los dos últimos valores.



2.2 Notación

Símbolo	Significado
k	número de iteración, empieza en 0
a	x_{k-1} (valor anterior)
b	x_k (valor actual)
x_{k+1}	valor nuevo

2.3 Fórmula

La recta que pasa por (a, f(a)) y (b, f(b)) es y = f(b) + [(f(b) − f(a)) / (b − a)]·(x − b). Igualando y = 0 y despejando:

```
            a · f(b) − b · f(a)
x(k+1) = --------------------
               f(b) − f(a)
```

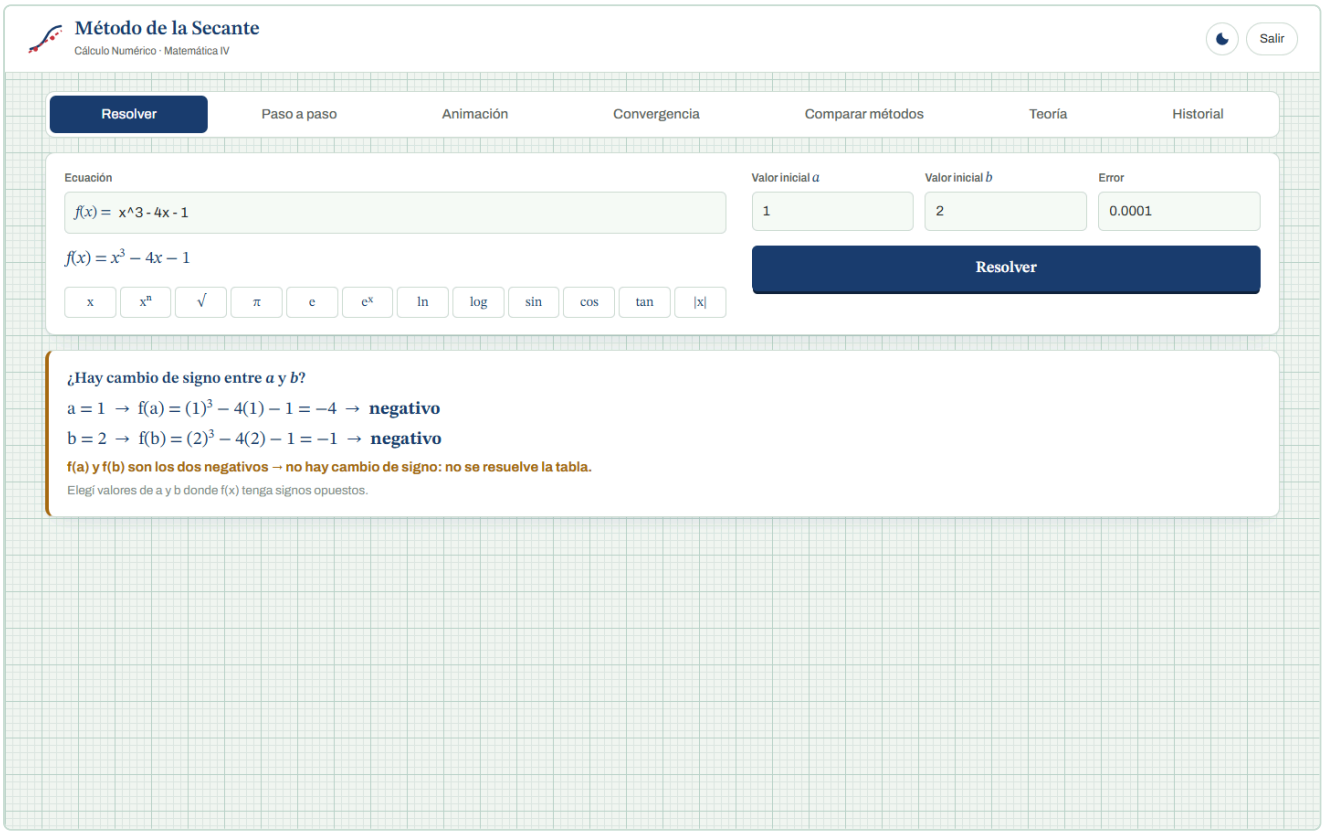
Es equivalente a $x_{k+1} = b - f(b)\cdot(b - a) / (f(b) - f(a))$, la forma de Newton-Raphson con la derivada reemplazada por la pendiente de la secante. Por eso **no necesita la derivada**.

2.4 Condición previa: cambio de signo

Antes de iterar se verifica que f(a) y f(b) tengan signos opuestos:

```
acheck ← f(a) > 0
bcheck ← f(b) > 0
si acheck = bcheck → no hay cambio de signo: NO se resuelve
```

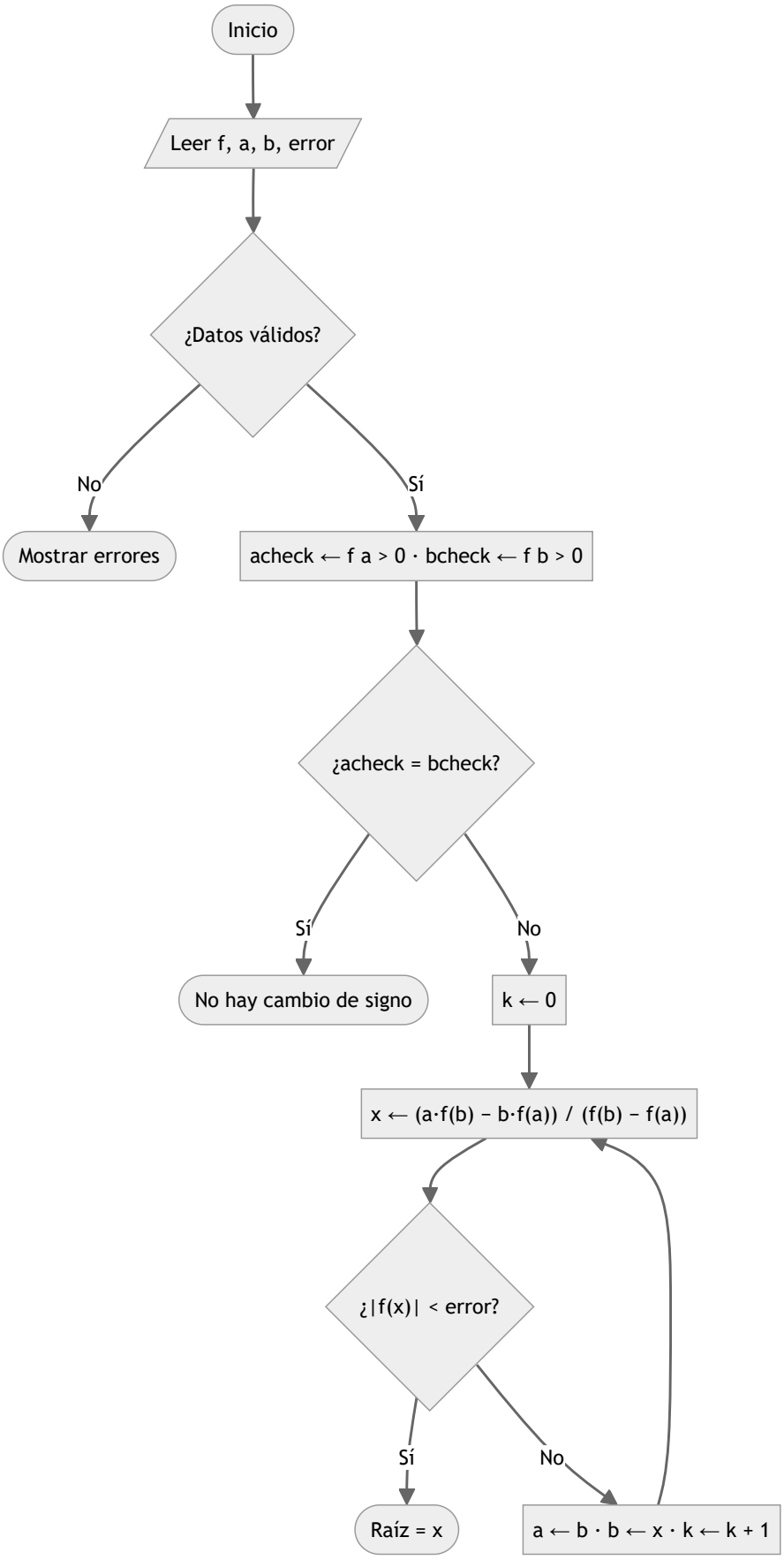
Si f es continua y hay cambio de signo, el **teorema de Bolzano** garantiza al menos una raíz entre a y b. En teoría la secante no exige esta condición (es un método abierto), pero el sistema la pide para asegurar que la raíz buscada esté entre los valores iniciales.



2.5 Algoritmo (pseudocódigo)

```
Leer f, a, b, error
acheck ← f(a) > 0
bcheck ← f(b) > 0
Si acheck = bcheck Entonces
    Escribir "No hay cambio de signo"
SiNo
    k ← 0
    encontrada ← Falso
    Mientras no encontrada y k < 100 Hacer
        x ← (a·f(b) − b·f(a)) / (f(b) − f(a))
        Si |f(x)| < error Entonces
            encontrada ← Verdadero
        FinSi
        a ← b
        b ← x
        k ← k + 1
    FinMientras
    Escribir "Raíz: ", x
FinSi
```

2.6 Diagrama de flujo



2.7 Criterio de parada y error

- **Test de parada:** se reemplaza x_{k+1} en la función. Si $\mathbf{|f(x_{k+1})| < error}$, x_{k+1} es la raíz. Se usa el valor absoluto porque f puede ser negativa.
- **Sin máximo de iteraciones elegido por el usuario:** se itera hasta llegar al error pedido. Hay un límite interno de 100 iteraciones que solo actúa si el método no converge, para que el programa no quede trabado.
- **Otros cortes:** si f(a) = f(b) (la fórmula dividiría por cero) o si x_{k+1} cae fuera del dominio de f.

2.8 Criterio de convergencia

Si f′(r) ≠ 0 y los valores iniciales están cerca de la raíz, el método converge con orden $\boldsymbol{\varphi = (1 + \sqrt{5})/2 \approx 1.618}$ (convergencia superlineal): en cada iteración la cantidad de cifras correctas se multiplica por ≈ 1.6. Es más rápido que bisección (orden 1) y algo más lento que Newton (orden 2).

2.9 Ventajas y limitaciones

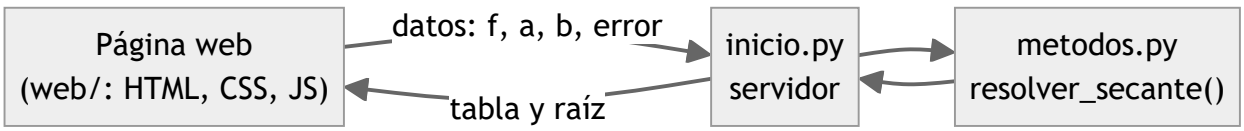
Ventajas - No necesita la derivada. - Converge rápido (orden ≈ 1.618). - Una sola evaluación nueva de f por iteración.

Limitaciones - Si f(a) = f(b), divide por cero. - Si a y b están lejos de la raíz, puede no converger. - Una asíntota entre a y b produce un cambio de signo sin raíz (ej.: 1/x en [−1, 2]). El sistema avisa si la raíz encontrada queda fuera de [a, b]. - En raíces múltiples converge lento.

---

3. Diseño del sistema

3.1 Arquitectura



- `inicio.py` **:** se ejecuta al inicio. Prende un servidor web local, abre el navegador y, cuando se toca Resolver, llama a `metodos.resolver_secante()`.
- `metodos.py` **:** todos los métodos del cálculo, escritos paso a paso.
- `web/` **:** la página. Muestra la tabla y el test de parada, y hace en JavaScript el paso a paso, la animación, los gráficos, la comparación, el historial y la exportación.

3.2 Estructura de archivos

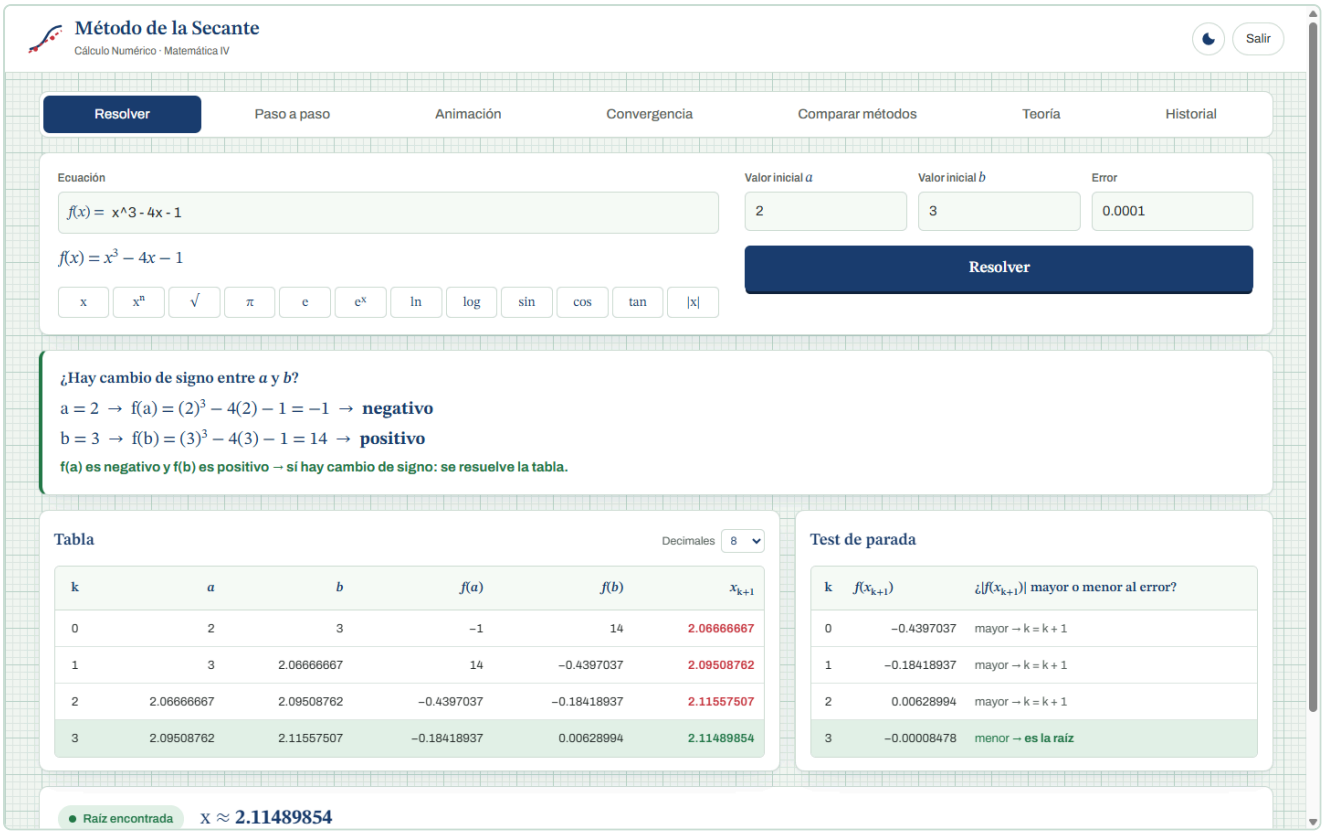
```
MATEMATICA/
├── inicio.py            Se ejecuta al inicio: servidor + llama a los métodos
├── metodos.py           Todos los métodos (validar, cambio de signo, fórmula, tabla)
├── construir_exe.py     Genera el ejecutable
├── tests/
│   └── test_metodos.py  Pruebas de metodos.py
├── web/                 La página
│   ├── index.html
│   ├── css/estilos.css
│   └── js/              app.js, pasos.js, animacion.js, comparacion.js, …
├── ejecutable/          MetodoSecante.exe
└── docs/                Documentación y capturas
```

3.3 Métodos de `metodos.py`

Método	Qué hace
`preparar_funcion(texto)`	Convierte lo que escribe el usuario a Python: `x^3 - 4x` → `x**3-4*x`
`funcion_esta_bien_escrita(funcion)`	Revisa que la función sea válida y dependa de x
`evaluar(funcion, x)`	Calcula f(x) reemplazando x por el número
`leer_numero(texto)`	Convierte texto en número (acepta coma decimal)
`validar_datos(...)`	Arma la lista de errores de los datos ingresados
`hay_cambio_de_signo(fa, fb)`	acheck = f(a) > 0, bcheck = f(b) > 0; iguales → no hay cambio
`calcular_x_siguiente(a, b, fa, fb)`	La fórmula (a·f(b) − b·f(a)) / (f(b) − f(a))
`resolver_secante(...)`	El método principal: valida, verifica el signo e itera armando la tabla

3.4 Diseño de la interfaz

Pestaña	Contenido
Resolver	Ecuación, a, b y error. Al resolver: el cambio de signo (a → f(a) → positivo o negativo, y lo mismo con b), la **tabla** (k, a, b, f(a), f(b), x_{k+1}) y al lado el **test de parada** (f(x_{k+1}) y la comparación con el error).
Paso a paso	La resolución de la tabla como la haría una persona: en cada iteración, la fórmula, el reemplazo y el resultado de f(a), f(b), x_{k+1} y del test, con las celdas que se usan pintadas de colores.
Animación	El método sobre la gráfica: la secante, su corte con el eje x y el test de cada iteración.
Comparar métodos	Secante contra Newton-Raphson, falsa posición y bisección.
Teoría	Fórmula, algoritmo, ventajas y limitaciones.
Historial	Ejercicios resueltos (guardados en el navegador).



k	a	b	f(a)	f(b)	x_{k+1}
0	2	3	−1	14	2.06666667
1	3	2.06666667	14	−0.4397037	2.09508762
2	2.06666667	2.09508762	−0.4397037	−0.18418937	2.11557507
3	2.09508762	2.11557507	−0.18418937	0.00628994	2.11489854

k	$f(x_{k+1})$	$\lvert f(x_{k+1})\rvert$ mayor o menor al error?
0	−0.4397037	mayor → k = k + 1
1	−0.18418937	mayor → k = k + 1
2	0.00628994	mayor → k = k + 1
3	−0.00008478	menor → es la raíz

4. Implementación

4.1 Cambio de signo

```
def hay_cambio_de_signo(fa, fb):
    acheck = fa > 0          # ¿f(a) es positivo?
    bcheck = fb > 0          # ¿f(b) es positivo?

    if acheck == bcheck:     # los dos positivos o los dos negativos
        return False         # -> NO hay cambio de signo
    else:
        return True          # -> SÍ hay cambio de signo
```

4.2 La fórmula

```
def calcular_x_siguiente(a, b, fa, fb):
    return (a * fb - b * fa) / (fb - fa)
```

4.3 Las iteraciones

```
k = 0
encontrada = False
while not encontrada and k < LIMITE_DE_ITERACIONES:
    fa = evaluar(funcion, a)
    fb = evaluar(funcion, b)
    x = calcular_x_siguiente(a, b, fa, fb)

    fx = evaluar(funcion, x)          # test de parada: reemplazar x(k+1) en la función
    if abs(fx) < error:
        encontrada = True

    respuesta["tabla"].append({"k": k, "a": a, "b": b, "fa": fa, "fb": fb, "x": x, "fx": fx,
...})

    a = b                             # siguiente iteración
    b = x
    k = k + 1
```

4.4 Decisiones de implementación

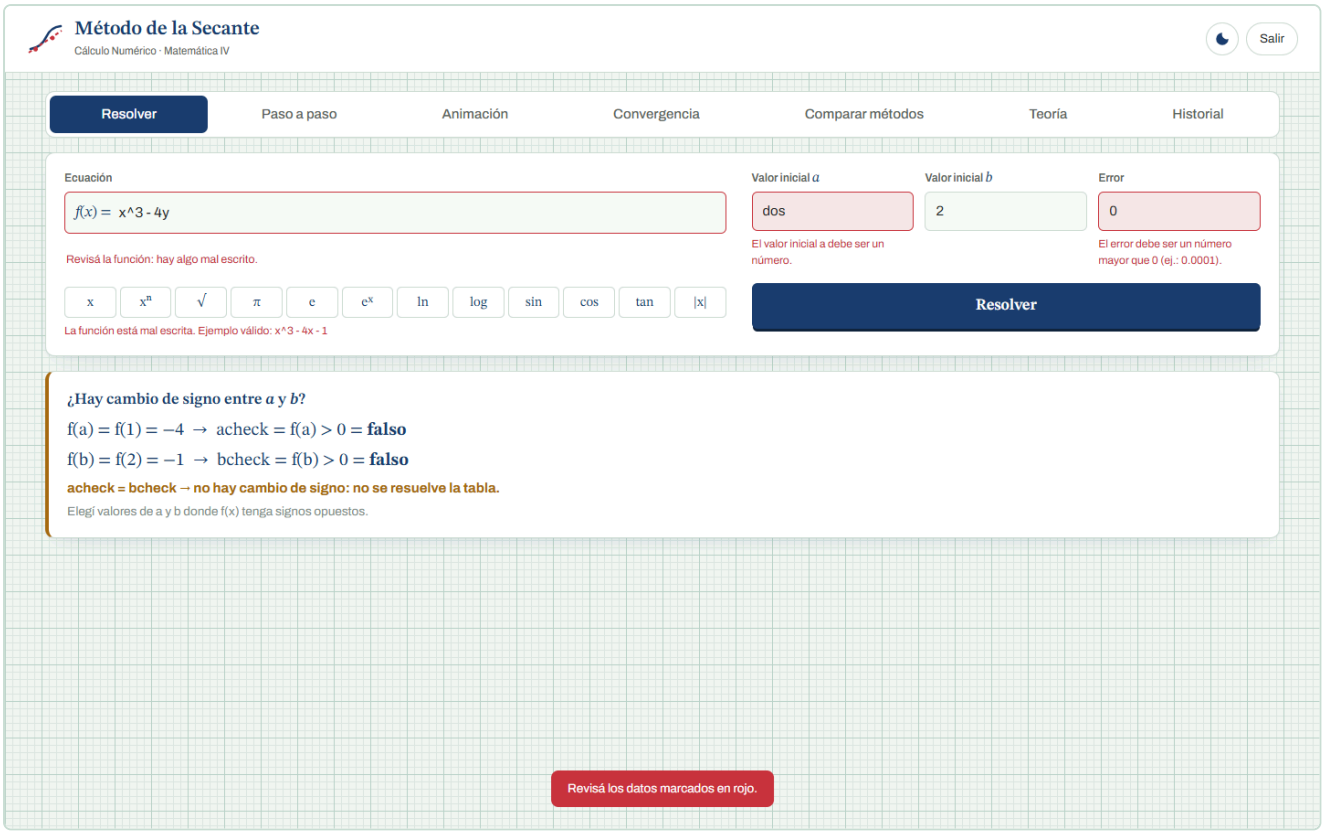
Decisión	Por qué
Test de parada $\|f(x_{k+1})\| < error$	Es el criterio visto en clase: se reemplaza x_{k+1} en la función y se mira qué tan cerca de 0 queda. Mide directamente lo que se busca (f(x) = 0) y no necesita conocer la raíz. Se usa el valor absoluto porque f puede quedar negativa. En la sección 6.4 se analiza cuánto se aleja x de la raíz real con este criterio.
Verificar el cambio de signo antes de iterar	La secante no lo exige, pero así el teorema de Bolzano garantiza que hay una raíz entre a y b, y el resultado es la raíz que el usuario está buscando y no otra lejana. Si no hay cambio de signo, el sistema avisa y no arma la tabla.
Si a o b ya son raíz, no se itera	Si f(a) = 0 o f(b) = 0, la respuesta ya está y no hace falta armar la tabla. Se muestra directamente cuál de los dos es la raíz.
Controlar f(a) = f(b)	La fórmula divide por f(b) − f(a). Antes de calcular x_{k+1} se revisa que no sea 0 y se muestra un mensaje, en vez de dejar que el programa falle.
Límite interno de 100 iteraciones	El usuario no elige un máximo de iteraciones: solo ingresa f, a, b y el error, como en clase. Pero si el método no converge, el ciclo no terminaría nunca. Como la secante converge con orden ≈ 1.618, un problema normal necesita menos de 10 iteraciones (sección 6.4). Llegar a 100 indica que no converge.
Avisar si la raíz queda fuera de [a, b]	Una asíntota (por ejemplo 1/x en [−1, 2]) produce un cambio de signo sin raíz. Si el resultado queda fuera del intervalo inicial, el sistema lo advierte.
x_{k+1} infinito o fuera del dominio	Si la fórmula da infinito, o f no se puede calcular en x_{k+1} (por ejemplo ln de un negativo), se corta con un mensaje claro en vez de mostrar un error de Python.
f(x) se calcula con Python, el método no	Python solo se usa como calculadora para obtener f(x): `eval` con una lista cerrada de nombres (x, sin, cos, ln, sqrt, e, pi…). Cualquier otro nombre se rechaza. La fórmula, las iteraciones y el test de parada están programados a mano en `metodos.py`, como pide la guía.

Decisión	Por qué
Escritura natural de la función	El usuario escribe como en el cuaderno: `x^3 - 4x - 1`, `2(x+1)`, coma decimal. `preparar_funcion` lo traduce a Python (`x**3-4*x-1`).
Cálculo en Python, interfaz web	El método queda en un archivo corto y fácil de leer (`metodos.py`). La página permite la tabla, el paso a paso, la animación y los gráficos sin instalar nada. Con PyInstaller todo queda en un solo `.exe` .

5. Validación de datos

Situación	Ejemplo	Mensaje
Datos incompletos	función vacía	"Falta escribir la función."
Función mal escrita	`x^3 - 4y`	"La función está mal escrita. Ejemplo válido: x^3 - 4x - 1"
Valores no numéricos	a = "dos"	"El valor inicial a debe ser un número."
Intervalo incorrecto	a = b	"a y b tienen que ser distintos."
Error inválido	0 o −1	"El error debe ser un número mayor que 0 (ej.: 0.0001)."
Fuera del dominio	ln(x) con a = −1	"La función no se puede calcular en a o en b (fuera de su dominio)."
Sin cambio de signo	a = 1, b = 2 en $x^3 - 4x - 1$	"f(a) y f(b) son los dos negativos → no hay cambio de signo: no se resuelve la tabla."
División por cero	f(a) = f(b)	"f(a) y f(b) son iguales: no se puede dividir por cero."
No converge	—	"Después de 100 iteraciones no se llegó al error pedido: el método no converge."
Asíntota	1/x en [−1, 2]	"Ojo: la raíz encontrada está fuera de [a, b]..."

El programa nunca se cierra por un dato mal ingresado: muestra los errores en rojo junto a cada campo.



6. Casos de prueba y resultados

6.1 Ejemplo resuelto a mano: f(x) = x³ − 4x − 1, a = 2, b = 3, error = 0.0001

- **Cambio de signo:** f(2) = −1 (acheck = falso), f(3) = 14 (bcheck = verdadero) → hay cambio de signo.
- **k = 0:** $x_1 = (2\cdot 14 - 3\cdot(-1)) / (14 - (-1)) = 31/15 =$ **2.06666667** ✓ (coincide con el sistema)

k	a	b	f(a)	f(b)	x_{k+1}	$f(x_{k+1})$	¿|f| < error?
0	2	3	−1	14	2.06666667	−0.4397037	no
1	3	2.06666667	14	−0.4397037	2.09508762	−0.18418937	no
2	2.06666667	2.09508762	−0.4397037	−0.18418937	2.11557507	0.00628994	no
3	2.09508762	2.11557507	−0.18418937	0.00628994	2.11489854	−0.00008478	**sí**

Raíz ≈ 2.11489854 (k = 3).

> El enunciado propone el intervalo [1, 2], pero ahí f(1) = −4 y f(2) = −1 tienen el mismo signo (acheck = bcheck = falso): **no hay cambio de signo** y el sistema no resuelve. La raíz (2.1149…) está fuera de [1, 2].

6.2 Otros casos

f(x)	a, b	Error	Resultado
cos(x) − x	0, 1	0.0001	Raíz ≈ 0.7391
e^(−x) − x	0, 1	0.000001	Raíz ≈ 0.5671
$x^2 - 2$	1, 2	10^{-10}	Raíz ≈ 1.41421356 (√2)
$x^2 - 4$	2, 5	0.01	a = 2 ya es raíz: f(2) = 0
$x^3 - 4x - 1$	1, 2	0.0001	No hay cambio de signo: no se resuelve
1/x	−1, 2	0.0001	Advertencia: la "raíz" queda fuera de [a, b] (asíntota)
ln(x)	−1, 2	0.1	Error: fuera del dominio

6.3 Verificación con valores de referencia

Los resultados del sistema se compararon con el valor exacto de cada raíz, obtenido sin usar el método de la secante:

- **$x^3 - 4x - 1$:** fórmula de Cardano (forma trigonométrica, porque tiene tres raíces reales): r = 2·√(4/3)·cos(⅓·arccos(3√3/16)) = 2.1149075414…
- **cos(x) − x:** número de Dottie, 0.7390851332…
- **e^(−x) − x:** constante omega, Ω = W(1) = 0.5671432904…
- **$x^2 - 2$:** √2 = 1.4142135624…

Los valores de referencia se pueden confirmar en GeoGebra (comando `Raíz(f, a, b)`) o en WolframAlpha.

f(x)	a, b	Error	Iteraciones	Raíz del sistema	Valor de referencia	|raíz − referencia|	|f(raíz)|
$x^3 - 4x - 1$	2, 3	10^{-4}	4	2.11489854	2.11490754	9.0×10^{-6}	8.5×10^{-5}
cos(x) − x	0, 1	10^{-4}	3	0.73911936	0.73908513	3.4×10^{-5}	5.7×10^{-5}
e^(−x) − x	0, 1	10^{-6}	4	0.56714331	0.56714329	1.6×10^{-8}	2.5×10^{-8}
$x^2 - 2$	1, 2	10^{-10}	6	1.41421356	1.41421356	$< 10^{-15}$	4.4×10^{-16}

En todos los casos se cumple el test de parada (|f(raíz)| < error) y la raíz coincide con el valor de referencia en las cifras que garantiza el error pedido.

6.4 Análisis del error y de la convergencia

Velocidad de convergencia. Se resolvió $f(x) = x^3 - 4x - 1$ con a = 2 y b = 3, pero con un error muy chico (10^{-12}) para ver más iteraciones. $e_k = |x_{k+1} - r|$ es la distancia a la raíz exacta r = 2.11490754147676:

k	x_{k+1}	$e_k = \lvert x_{k+1} - r \rvert$	$\lvert f(x_{k+1}) \rvert$	Orden estimado p
0	2.066666666666667	4.8×10^{-2}	4.4×10^{-1}	—
1	2.095087618500431	2.0×10^{-2}	1.8×10^{-1}	—
2	2.115575069066892	6.7×10^{-4}	6.3×10^{-3}	3.81
3	2.114898540057763	9.0×10^{-6}	8.5×10^{-5}	1.27
4	2.114907537430406	4.0×10^{-9}	3.8×10^{-8}	1.79
5	2.114907541476780	2.5×10^{-14}	2.3×10^{-13}	1.56

El orden se estima con $p \approx \ln(e_k / e_{k-1}) / \ln(e_{k-1} / e_{k-2})$. En las primeras iteraciones salta porque los valores todavía están lejos de la raíz. A medida que se acercan, p se estabiliza alrededor de **1.6**, que coincide con el orden teórico $\varphi \approx 1.618$ (sección 2.8). Se ve en la tabla: el error pasa de 10^{-4} a 10^{-6}, 10^{-9} y 10^{-14}, y las cifras correctas se multiplican por ≈ 1.6 en cada paso. Por eso alcanzan 6 iteraciones para llegar a 10^{-12}.

Qué garantiza el test de parada. El sistema corta cuando $|f(x)| <$ error, pero eso no es lo mismo que $|x - r| <$ error. Cerca de la raíz, $f(x) \approx f'(r) \cdot (x - r)$, entonces:

$$|x - r| \approx \frac{|f(x)|}{|f'(r)|}$$

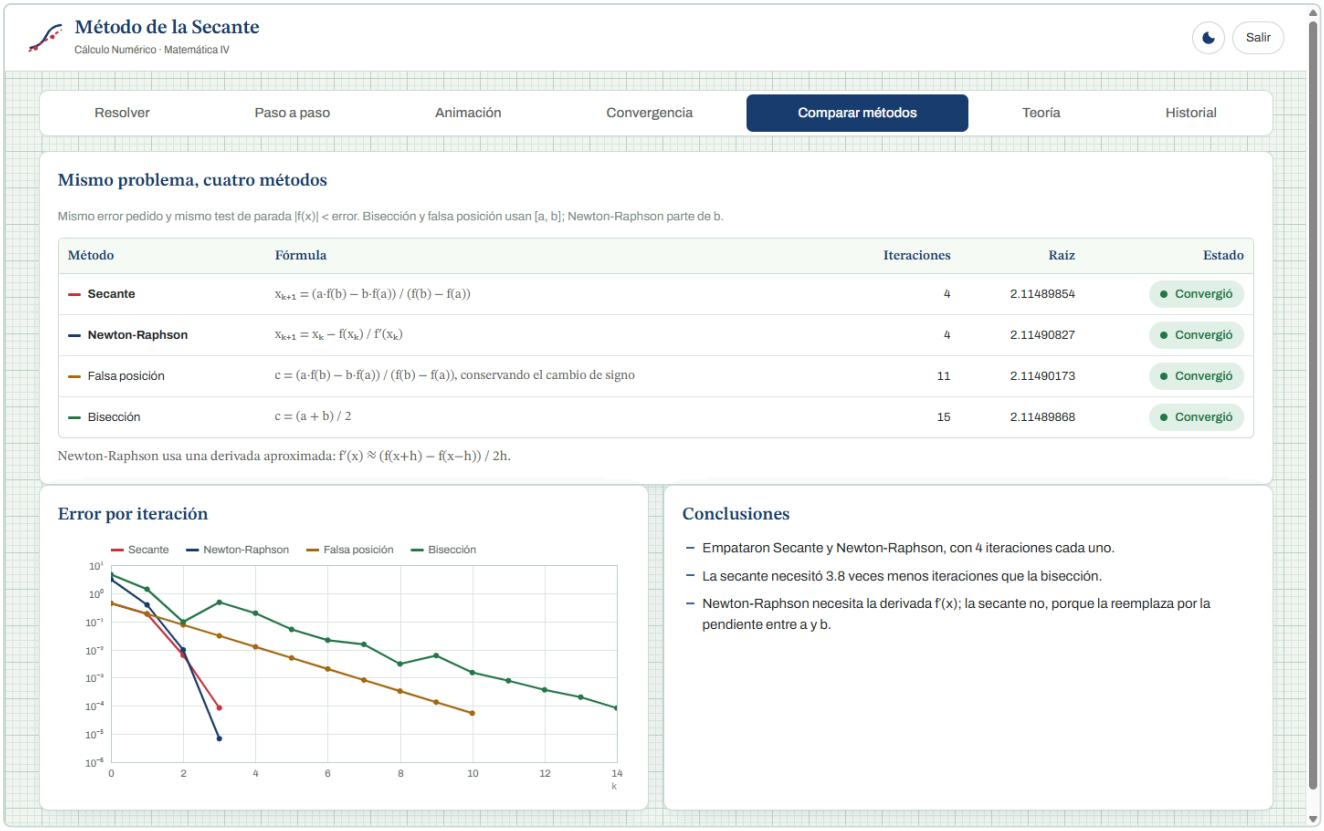
- En $x^3 - 4x - 1$, $f'(r) = 3r^2 - 4 \approx 9.42$. Con $|f| = 8.5 \times 10^{-5}$, la distancia a la raíz es $\approx 8.5 \times 10^{-5} / 9.42 \approx 9.0 \times 10^{-6}$, igual a la diferencia medida en la sección 6.3.
- En $\cos(x) - x$, $|f'(r)| = \sin(r) + 1 \approx 1.67$, así que $5.7 \times 10^{-5} / 1.67 \approx 3.4 \times 10^{-5}$, también igual a lo medido.

Cuando $|f'(r)| > 1$ (la función corta el eje con pendiente pronunciada), la raíz es incluso más precisa que el error pedido. Cuando la función es muy plana cerca de la raíz ($|f'(r)| < 1$, o raíces múltiples con $f'(r) = 0$), el valor de x puede quedar más lejos de la raíz de lo que sugiere el error. Esta es una limitación del criterio, y por eso conviene pedir un error más chico en esos casos.

6.5 Pruebas automáticas

`python -m unittest discover -s tests -t .` ejecuta las 10 pruebas de `tests/test_metodos.py`: conversión de la función, f(2) = −1, cambio de signo, la fórmula a mano (31/15), el ejemplo completo, el corrimiento de a y b, la falta de cambio de signo, las validaciones y la advertencia de asíntota.

6.6 Comparación con otros métodos (mismo problema)



Con el mismo test de parada |f(x)| < 0.0001: la secante necesitó 4 iteraciones, Newton-Raphson 4, falsa posición 11 y bisección 15.

---

7. Funcionalidades adicionales

Sugerida en la guía	Implementación
Representación gráfica	Pestaña Animación: la función y las rectas secantes, con zoom y desplazamiento
Comparación automática	Secante, Newton-Raphson, falsa posición y bisección
Visualización del error	Cuadro del test de parada y paso a paso con colores
Exportación	Excel (CSV), texto e informe imprimible como PDF
Historial	Ejercicios resueltos, guardados en el navegador
Interfaz avanzada	Paso a paso animado, modo claro y oscuro, adaptable a celular

---

8. Manual de uso

1. Abrir el programa: doble clic en `ejecutable/MetodoSecante.exe`, o `python inicio.py`.

2. Escribir la **ecuación** (por ejemplo `x^3 - 4x - 1`; se puede usar `sin`, `cos`, `ln`, `log`, `sqrt`, `e`, `pi`).
3. Escribir los valores iniciales **a** y **b**, de modo que f cambie de signo entre ellos.
4. Escribir el **error** (por ejemplo 0.0001).
5. Tocar **Resolver**. Aparecen el cambio de signo, la tabla y el test de parada.
6. Opcional: pestañas Paso a paso, Animación, Comparar métodos, Teoría e Historial.
7. Para cerrar: botón **Salir**.